

Netzplan App – Projektmanagement mit Netzplantechnik / Project Management with Critical Path Method

Entwickelt im Rahmen einer Umschulung als Portfolioprojekt.
Developed as a portfolio project during vocational retraining.

Deutsch

Projektbeschreibung

Diese Anwendung ist ein vollständig funktionsfähiges **Projektmanagement-System auf Basis der Netzplantechnik (CPM – Critical Path Method)**. Sie ermöglicht die strukturierte Planung, Verwaltung und Auswertung von Projekten mit hierarchischen Aufgaben, automatischer CPM-Berechnung und visueller Darstellung als Netzplan und Gantt-Diagramm.

Das System wurde von Grund auf selbst entwickelt – als installierbare Desktop- und Tablet-Anwendung mit zentralem Server-Backend, Nutzerverwaltung und Rollenkonzept.

Funktionsumfang

- **Netzplan-Editor** – Aufgaben als Knoten visuell darstellen, verschieben und verknüpfen
- **Automatische CPM-Berechnung** – FAZ, FEZ, SAZ, SEZ, GP und FP werden automatisch berechnet und aktualisiert
- **Kritischer Pfad** – Visuell hervorgehoben durch rötlich hinterlegte Rasterzeile und farbige Verbindungslinien
- **Hierarchische Aufgabenstruktur** – Aufgaben bis zu 6 Ebenen tief mit Teilaufgaben gliedern (Work Breakdown Structure)
- **Fortschrittsanzeige** – Prozentualer Fortschritt je Ebene, automatisch berechnet aus erledigten Teilaufgaben
- **Status-Workflow** – Offen → In Bearbeitung → Prüfen → Erledigt, mit automatischer Farbkodierung
- **Gantt-Diagramm** – Automatisch generiert aus den Netzplan-Daten, responsiv skalierend
- **Zoom-Funktion** – Netzplan frei skalierbar per Slider
- **Export** – Netzplan und Gantt als PNG und PDF exportierbar
- **Nutzerverwaltung** – Rollenkonzept mit Admin, Projektmanager und Mitarbeiter
- **Multi-Projekt** – Mehrere Projekte parallel, Nutzer sehen nur ihre eigenen
- **Server-Backend** – Zentrale Datenhaltung, selbst-hostbar per Docker
- **Cross-Platform** – Läuft auf Windows, macOS, Linux, Android (Kivy)

Architektur & Module

Backend

Datei	Aufgabe
<code>backend/main.py</code>	FastAPI-Server – Startpunkt, Middleware, Login-Endpunkt
<code>backend/database.py</code>	Datenbankverbindung (PostgreSQL via SQLAlchemy)
<code>backend/models.py</code>	Datenmodelle: User, Project, Task, TaskDependency, Comment
<code>backend/auth.py</code>	Login, JWT-Token, Passwort-Hashing, Rollenprüfung
<code>backend/routers/users.py</code>	API-Endpunkte für Nutzerverwaltung
<code>backend/routers/projects.py</code>	API-Endpunkte für Projekte und Mitgliedschaft
<code>backend/routers/tasks.py</code>	API-Endpunkte für Aufgaben, CPM-Berechnung, Kommentare
<code>backend/create_admin.py</code>	Skript zum Anlegen des ersten Admin-Nutzers

Frontend

Datei	Aufgabe
frontend/main.py	Kivy-App – Startpunkt, Screen-Manager
frontend/screens/login_screen.py	Login-Maske
frontend/screens/dashboard_screen.py	Projektübersicht, Nutzerverwaltung
frontend/screens/project_screen.py	Projektdetails, Navigation
frontend/screens/network_screen.py	Netzplan-Editor mit Drag & Drop, Zoom, Export
frontend/screens/gantt_screen.py	Gantt-Diagramm mit Export
frontend/screens/users_screen.py	Nutzerverwaltung (Admin)
frontend/widgets/node_widget.py	Netzplan-Knoten mit Kontextmenü und Touch-Events
frontend/widgets/tooltip_widget.py	Hover-Tooltip mit Fortschrittsbalken
frontend/widgets/progress_bar.py	Wiederverwendbarer Fortschrittsbalken
shared/constants.py	Gemeinsame Konstanten: Rollen, Status, Farben, API-URL

Rollen & Berechtigungen

Rolle	Rechte
Admin / Projektleiter	Vollzugriff: Nutzer, Projekte, Aufgaben, alle Daten
Projektmanager	Vollzugriff innerhalb eigener Projekte
Mitarbeiter	Eigene Aufgaben: Status setzen, Kommentare, Fortschritt

Installation & Start

Voraussetzungen:

- Python 3.11 oder höher
- Docker Desktop
- pip

Abhängigkeiten installieren:

```
bash
```

```
pip install fastapi uvicorn sqlalchemy psycpg2-binary passlib python-jose kivy requests
```

Datenbank und Backend starten:

```
bash
```

```
docker-compose up -d
```

Ersten Admin-Nutzer anlegen:

```
bash
```

```
docker exec -it netzplan_backend python create_admin.py
```

Frontend starten:

```
bash
```

```
python frontend/main.py
```

Selbst-Hosting (Server)

Die App ist vollständig selbst-hostbar. `docker-compose.yml` startet PostgreSQL und das FastAPI-Backend gemeinsam. Wer die App produktiv einsetzen möchte, benötigt einen eigenen Server mit Docker.

```
bash
```

```
docker-compose up -d
```

API läuft dann unter: `http://<server-ip>:8000`

Frontend-URL in `shared/constants.py` anpassen:

```
python
```

```
API_BASE_URL = "http://<server-ip>:8000"
```

Tech-Stack

Bereich	Technologie
Frontend	Python · Kivy
Backend	Python · FastAPI · Uvicorn
Datenbank	PostgreSQL · SQLAlchemy
Auth	JWT · passlib · bcrypt
Export	ReportLab (PDF) · Kivy Canvas (PNG)
Hosting	Docker · Docker Compose

English

Project Description

This application is a fully functional **project management system based on the Critical Path Method (CPM)**. It enables structured planning, management and evaluation of projects with hierarchical tasks, automatic CPM calculation and visual representation as a network diagram and Gantt chart.

The system was developed entirely from scratch - as an installable desktop and tablet application with a central server backend, user management and a role-based access concept.

Features

- **Network diagram editor** - Visualise, move and link tasks as nodes
- **Automatic CPM calculation** - ESD, EFD, LSD, LFD, TF and FF are calculated and updated automatically
- **Critical path** - Visually highlighted by a reddish grid row and coloured connection lines
- **Hierarchical task structure** - Break tasks down up to 6 levels deep with subtasks (Work Breakdown Structure)
- **Progress tracking** - Percentage progress per level, automatically calculated from completed subtasks
- **Status workflow** - Open → In Progress → Review → Done, with automatic colour coding
- **Gantt chart** - Automatically generated from network diagram data, responsively scaling

- **Zoom function** - Network diagram freely scalable via slider
 - **Export** - Network diagram and Gantt exportable as PNG and PDF
 - **User management** - Role-based concept with Admin, Project Manager and Team Member
 - **Multi-project** - Multiple projects in parallel, users only see their own
 - **Server backend** - Centralised data storage, self-hostable via Docker
 - **Cross-platform** - Runs on Windows, macOS, Linux, Android (Kivy)
-

Architecture & Modules

Backend

File	Purpose
<code>backend/main.py</code>	FastAPI server - entry point, middleware, login endpoint
<code>backend/database.py</code>	Database connection (PostgreSQL via SQLAlchemy)
<code>backend/models.py</code>	Data models: User, Project, Task, TaskDependency, Comment
<code>backend/auth.py</code>	Login, JWT token, password hashing, role checks
<code>backend/routers/users.py</code>	API endpoints for user management
<code>backend/routers/projects.py</code>	API endpoints for projects and membership
<code>backend/routers/tasks.py</code>	API endpoints for tasks, CPM calculation, comments
<code>backend/create_admin.py</code>	Script to create the first admin user

Frontend

File	Purpose
frontend/main.py	Kivy app - entry point, screen manager
frontend/screens/login_screen.py	Login screen
frontend/screens/dashboard_screen.py	Project overview, user management
frontend/screens/project_screen.py	Project details, navigation
frontend/screens/network_screen.py	Network diagram editor with drag & drop, zoom, export
frontend/screens/gantt_screen.py	Gantt chart with export
frontend/screens/users_screen.py	User management (admin)
frontend/widgets/node_widget.py	Network node with context menu and touch events
frontend/widgets/tooltip_widget.py	Hover tooltip with progress bar
frontend/widgets/progress_bar.py	Reusable progress bar
shared/constants.py	Shared constants: roles, status, colours, API URL

Roles & Permissions

Role	Permissions
Admin / Project Lead	Full access: users, projects, tasks, all data
Project Manager	Full access within own projects
Team Member	Own tasks: set status, comments, view progress

Installation & Start

Requirements:

- Python 3.11 or higher
- Docker Desktop
- pip

Install dependencies:

```
bash
pip install fastapi uvicorn sqlalchemy psycopg2-binary passlib python-jose kivy requ
```

Start database and backend:

```
bash
docker-compose up -d
```

Create first admin user:

```
bash
docker exec -it netzplan_backend python create_admin.py
```

Start frontend:

```
bash
python frontend/main.py
```

Self-Hosting

The app is fully self-hostable. `docker-compose.yml` starts PostgreSQL and the FastAPI backend together. Anyone wishing to use the app in production needs their own server with Docker.

```
bash
docker-compose up -d
```

API runs at: `http://<server-ip>:8000`

Update the frontend URL in `shared/constants.py`:

```
python
API_BASE_URL = "http://<server-ip>:8000"
```

Tech Stack

Area	Technology
Frontend	Python · Kivy
Backend	Python · FastAPI · Uvicorn
Database	PostgreSQL · SQLAlchemy
Auth	JWT · passlib · bcrypt
Export	ReportLab (PDF) · Kivy Canvas (PNG)
Hosting	Docker · Docker Compose

Entwickelt mit Python 3 · FastAPI · Kivy · PostgreSQL · Docker
Developed with Python 3 · FastAPI · Kivy · PostgreSQL · Docker